

TECNOLOGÍAS Y ARQUITECTURA PARA EL BACKEND

1. Objetivo

Definir la arquitectura monolítica, el patrón de diseño MVC y las tecnologías backend que se utilizarán para el desarrollo del sistema 3Market, garantizando la persistencia de datos, la seguridad de la aplicación, el procesamiento de solicitudes web y la integración robusta con el motor de plantillas.

2. Arquitectura y Patrón de Diseño

Arquitectura Monolítica y MVC: Para el desarrollo del servidor de 3Market se ha seleccionado una arquitectura estructurada bajo las siguientes premisas:

- **Modelo-Vista-Controlador (MVC):** Permite separar la lógica de negocio y acceso a datos (Modelos y Servicios), el flujo de control de peticiones HTTP (Controladores) y la representación visual renderizada en el servidor mediante Thymeleaf (Vistas).
- **Arquitectura Monolítica:** Centraliza todos los módulos de la aplicación (gestión de productos, usuarios, seguridad y vistas) en un único ejecutable empaquetado, facilitando el despliegue rápido, la consistencia transaccional y la gestión unificada en entornos académicos y de producción inicial.

3. Tecnologías y Dependencias Seleccionadas

El proyecto se construira sobre Spring Boot 4.1.1 con Java 21, utilizando el siguiente conjunto de dependencias oficiales especificadas en la configuración de Maven:

- **Java 21:** Lenguaje de programación orientado a objetos, aprovechando las características modernas del JDK para un rendimiento óptimo y tipado seguro.
- **Spring Boot Starter WebMVC:** Proporciona el ecosistema MVC de Spring, controladores web (@Controller), gestión de mapeo HTTP, despacho de solicitudes y soporte completo para vistas dinámicas.
- **Spring Boot Starter Data JPA:** dependencia para la persistencia de datos simplificada la base de datos relacional mediante Hibernate, mapeo objeto-relacional (ORM) y repositorios JPA.
- **Spring Boot Starter Security:** Módulo de autenticación y control de acceso que protege los endpoints, gestiona sesiones de usuario y cifra las credenciales de acceso, permitiendo validar los roles y permisos para consumir las diferentes URI.
- **Thymeleaf & Thymeleaf Extras SpringSecurity6:** Motor de plantillas del lado del servidor para procesar archivos HTML dinamicos.

- **Spring Boot Starter Validation:** Permite validar los datos de entrada en formularios y DTOs mediante anotaciones estándar (como @NotNull, @Size, @Email), asegurando la que los datos de entradas sean lo corrector para ser procesados.
- **Drive MySQL:** Driver oficial JDBC para conectar la aplicación Spring Boot con bases de datos relacionales MySQL.
- **Lombok:** Librería de desarrollo para reducir el código repetitivo mediante anotaciones automáticas como @Data, @Getter, @Setter, @NoArgsConstructor y @AllArgsConstructor.
- **Spring Boot DevTools:** Herramientas de desarrollo que habilitan el reinicio automático del servidor y la recarga rápida de recursos estáticos y plantillas durante la fase de programación.

4. Estructura de Paquetes y Capas

El código fuente del backend se organiza bajo una estructura limpia orientada a capas dentro del paquete raíz com.unipacifico._3market:

- **config/:** Clases de configuración general del sistema (Beans de seguridad, beans globales, etc.).
- **controller/:** Controladores web encargados de recibir las peticiones HTTP del usuario, invocar servicios y retornar las vistas Thymeleaf.
- **dto/:** Objetos de transferencia de datos para estructurar la información entre las vistas y la lógica de negocio sin exponer directamente los modelos.
- **mapper/:** Componentes o clases de conversión para transformar entidades JPA a DTOs y viceversa.
- **model/:** Clases de entidad anotadas con @Entity que representan las tablas de la base de datos relacional.
- **repository/:** Interfaces de persistencia que extienden de JpaRepository para realizar operaciones CRUD automáticas contra MySQL.
- **security/:** Clases y configuraciones específicas de Spring Security, filtros de autenticación y manejo de sesiones.
- **service/:** Capa de servicios que encapsula las reglas de negocio y transacciones de la aplicación.

5. Alcance

Esta definición corresponde exclusivamente al desarrollo del backend y la lógica de procesamiento del servidor de 3Market bajo arquitectura MVC con Spring Boot. Incluye la configuración de la conexión a base de datos, el mapeo de entidades, la capa de seguridad y la integración con las vistas Thymeleaf.

6. Consideraciones Técnicas

Mantener una separación estricta de responsabilidades entre controladores, servicios y repositorios.

Asegurar el manejo adecuado de transacciones en la capa de servicios mediante anotaciones como @Transactional para asegurarse que todos los procesos se cumplan en la persistencia de datos.

Configurar la ruta de almacenamiento de archivos de productos fuera del código fuente (en directorios externos como “./imagen/products)!”.

Acatar el estándar de nombramiento en los paquetes en singular (controller, model, service, repository, config) según los acuerdos del equipo.

7. Estado

Configuración y especificación técnica aprobada para el desarrollo del backend de 3Market. La implementación debe ajustarse rigurosamente a las dependencias y arquitectura establecidas.